ENAE464 - 0202

Lab05: Ludwieg-Tube Experiments on a Free-Flying Cone

Due on April 17th, 2026 at 11:59 PM

Dr. Silbaugh, 02:00 PM

Mikołaj Kostrzewa & Vai Srivastava

Experiment Performed: April 10th, 2026
Report Submitted: April 15th, 2026

April 15th, 2026

Enclosed is the technical report for the *Ludwieg-Tube Experiments on a Free-Flying Cone* laboratory experiment. This report presents the experimental methodology, results, and analysis of flow characteristics and effects for a free-flying cone in a Mach-4 Ludwieg tube.
Please feel free to contact us with any questions regarding the contents of this report.

Respectfully,
Mikołaj Kostrzewa & Vai Srivastava

Contents

1 Abstract **1**

2 Introduction **1**
2.1 Theoretical Background . . . 1
2.2 Objectives . . . 2

3 Methodology **2**
3.1 Experimental Apparatus . . . 2
3.2 Operating Procedure . . . 3

4 Results **3**
4.1 Flow Velocity . . . 3
4.2 Reynolds Number . . . 3
4.3 Velocity Fluctuation . . . 4
4.4 Expected Shedding Frequency . . . 4
4.5 Stokes Number . . . 4

5 Analysis and Discussion **5**
5.1 Measured Shedding Frequency . . . 5
5.2 Comparison to Theory . . . 5
5.2.1 Limited Temporal Resolution . . . 5
5.2.2 Spatial Measurement Location . . . 5
5.2.3 Higher Harmonics and Flow Complexity . . . 5
5.2.4 Three-Dimensional Effects . . . 6

6 Summary and Conclusions **7**

7 References **8**

8 Appendices **8**
8.1 Data . . . 8
8.2 Code . . . 8

Figures

3.1.1 Schematic diagram of the PIV experimental setup showing the double-pulsed Nd:YAG laser, sheet optics, CCD camera, and seeded flow field illuminated by the laser sheet. . . . 2
4.3.1 Transverse velocity fluctuation at a point near the wake centerline as a function of time. . . . 4
5.2.1 Evolution of the velocity magnitude field at six time instants spanning the 0.38 s measurement period. . . . 6
5.2.2 Representative instantaneous velocity vector field at $t = 0.19$ s. . . . 7

Listings

8.2.1 PIV Data Analysis Script. . . . 8

1 Abstract

This report presents an experimental investigation of vortex shedding downstream of a circular cylinder using Particle Imaging Velocimetry (PIV). The experiment was conducted in a water tunnel at a pump frequency of 2.1 Hz, corresponding to an incoming flow velocity of 0.0351 m/s and a Reynolds number of approximately 1100. PIV measurements captured instantaneous velocity fields over a 0.38 s duration, revealing the characteristic Kármán vortex street pattern. From the Strouhal number of 0.21 for the experimental Reynolds number regime, the expected vortex shedding frequency was calculated to be 0.58 Hz. Transverse velocity fluctuations at the cylinder wake centerline were analyzed using sinusoidal curve fitting, yielding a measured dominant frequency of 2.49 Hz. The Stokes number for the 20 µm polyamide seed particles was found to be 3.03×10^{-5}, confirming that particles faithfully follow the flow dynamics. The discrepancy between expected and measured frequencies suggests the presence of higher-order flow phenomena or measurement artifacts, which are discussed in detail.

2 Introduction

Particle Imaging Velocimetry (PIV) is a non-intrusive optical flow diagnostic technique widely used in experimental fluid mechanics to obtain instantaneous, spatially-resolved velocity fields. Unlike point-measurement techniques such as hot-wire anemometry or Laser Doppler Velocimetry (LDV), PIV provides full-field velocity information across a planar cross-section of the flow, making it particularly valuable for visualizing and quantifying complex flow structures such as vortices, separation zones, and turbulent eddies.

The fundamental principle of PIV involves seeding the flow with tracer particles that are sufficiently small and neutrally-buoyant to faithfully follow fluid motion. A laser sheet illuminates a planar region of the seeded flow at two closely-spaced time intervals, and a high-resolution camera captures the scattered light from the particles at each instant. Cross-correlation algorithms applied to small interrogation windows within the image pairs yield displacement vector fields, which, when divided by the known time interval, provide velocity fields.

In this experiment, PIV is applied to investigate the classic fluid mechanics phenomenon of vortex shedding from a circular cylinder in cross-flow. This configuration is of both fundamental and practical importance, as it represents a canonical bluff-body wake flow with applications ranging from heat exchanger tube arrays to structural vibrations induced by wind or water currents.

2.1 Theoretical Background

When a circular cylinder is placed in a steady, uniform cross-flow at moderate Reynolds numbers ($Re = \frac{U_\infty D}{\nu}$, where U_∞ is the free-stream velocity, D is the cylinder diameter, and ν is the kinematic viscosity), the flow exhibits periodic vortex shedding downstream of the cylinder. This phenomenon, known as the Kármán vortex street, results from an instability in the separated shear layers on either side of the cylinder, causing alternating vortices to be shed into the wake.

The dimensionless shedding frequency is characterized by the Strouhal number:

$$St = \frac{f \cdot D}{U_\infty} \tag{2.1.1}$$

where f is the vortex shedding frequency. For circular cylinders in the Reynolds number range $10^2 < Re < 10^5$, the Strouhal number is approximately constant at $St \approx 0.20$–0.21, a value well-established through extensive experimental and computational studies.

The quality of PIV measurements depends critically on the fidelity with which seed particles follow the flow. This is quantified by the Stokes number:

$$St_k = \frac{\tau_r}{\tau_f} \tag{2.1.2}$$

where $\tau_r = \frac{\rho_p D_p^2}{18\mu}$ is the particle response time (ρ_p is particle density, D_p is particle diameter, μ is fluid dynamic viscosity), and τ_f is the characteristic flow time scale. For accurate flow tracking, $St_k \ll 1$ is required, typically $St_k < 0.1$.

2.2 Objectives

The specific objectives of this experiment are:

1. Measure the incoming water flow velocity and calculate the Reynolds number for flow over the circular cylinder
2. Determine the expected vortex shedding frequency using the Strouhal number correlation for the experimental Reynolds number
3. Calculate the Stokes number to assess the suitability of the seed particles for flow visualization
4. Extract transverse velocity fluctuations in the cylinder wake from PIV data and identify the dominant frequency through sinusoidal curve fitting
5. Compare the experimentally measured shedding frequency with the theoretical prediction
6. Visualize the instantaneous velocity fields to identify vortex structures in the wake

3 Methodology

3.1 Experimental Apparatus

The experiment was conducted in a recirculating water tunnel with a test section of 6 in height, 6 in width, and 12 in length. A circular cylinder of diameter $D = 0.0127\,\mathrm{m}$ (0.5 in) was mounted horizontally at mid-height, spanning the full width of the test section. The flow velocity was controlled by adjusting the pump frequency, with a calibrated relationship $U_\infty = 0.0185 \times f_{\mathrm{pump}} - 0.0038$, where U_∞ is in meters per second and f_{pump} is the pump frequency in Hertz.
The PIV system (LaVision Inc.) consisted of a double-pulsed Nd:YAG laser (wavelength 532 nm) and a high-resolution CCD camera positioned perpendicular to the laser sheet plane. Cylindrical lenses shaped the laser beam into a thin vertical sheet (100 µm to 1 mm thick) aligned with the cylinder centerline in the streamwise direction. The laser pulses and camera shutter were synchronized through LaVision software.

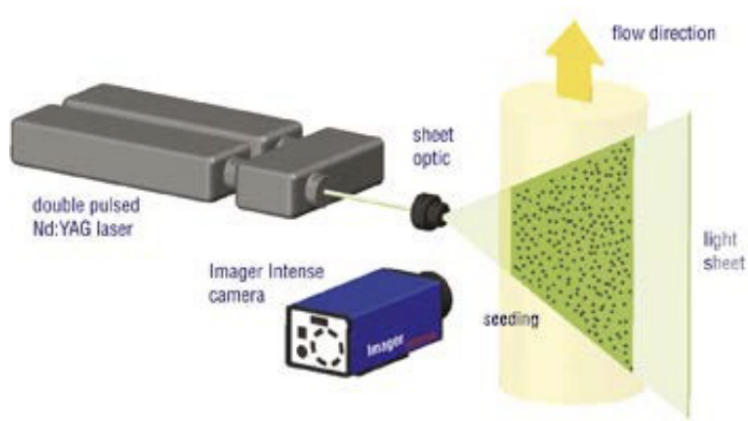

Figure 3.1.1: Schematic diagram of the PIV experimental setup showing the double-pulsed Nd:YAG laser, sheet optics, CCD camera, and seeded flow field illuminated by the laser sheet.

The flow was seeded with polyamide particles having mean diameter $D_p = 20\,\mu\text{m}$ and density $\rho_p = 950\,\text{kg/m}^3$. The seeding concentration was optimized to provide 7–10 particles per interrogation window, balancing the need for sufficient particle images with the requirement to avoid particle overlap and maintain spatial resolution.

3.2 Operating Procedure

The experimental procedure was as follows:

1. The water tunnel pump was started and adjusted to a frequency of 2.1 Hz, establishing a steady flow with incoming velocity $U_\infty = 0.0351\,\text{m/s}$.
2. Polyamide seed particles were introduced into the flow and allowed to reach uniform dispersion throughout the test section.
3. Laser safety goggles were worn by all personnel, and the laser system was activated under instructor supervision.
4. The PIV system acquired 20 image pairs over a total duration of 0.38 s, corresponding to a temporal resolution of approximately 20 ms between successive measurements.
5. Three different pulse separation times (Δt) were tested to optimize particle displacement for the cross-correlation algorithm.
6. Raw image pairs were processed using LaVision DaVis software with an interrogation window size of 32×32 pixels and 50 % overlap (vector spacing of 16 pixels).
7. Velocity vector fields were exported as ASCII data files for post-processing and analysis.

4 Results

4.1 Flow Velocity

The incoming water flow speed $\bar{U}_\infty$ was calculated from the pump frequency using the provided calibration relationship:

$$U_\infty = 0.0185 \times f_{\text{pump}} - 0.0038 = 0.0185 \times 2.1 - 0.0038 = 0.0351\,\frac{\text{m}}{\text{s}} \tag{4.1.1}$$

The PIV-measured mean velocity field confirmed this value, with spatial-average velocity components of $\bar{u} = 0.0245\,\text{m/s}$ in the streamwise direction and $\bar{v} = 0.0002\,\text{m/s}$ in the transverse direction. The streamwise velocity is lower than U_∞ because the spatial average includes the low-velocity wake region downstream of the cylinder. The near-zero mean transverse velocity indicates symmetric flow, as expected for this Reynolds number regime.

4.2 Reynolds Number

The Reynolds number for the water flow over the cylinder was calculated using the cylinder diameter as the characteristic length scale and the kinematic viscosity of water at room temperature (22 °C):

$$Re = \frac{U_\infty D}{\nu} = \frac{0.0351 \times 0.0127}{4.048 \times 10^{-7}} \approx 1100 \tag{4.2.1}$$

This Reynolds number falls well within the periodic vortex shedding regime ($Re \approx 50$–$200{,}000$) and specifically within the range where the Strouhal number is approximately constant at $St \approx 0.21$. At this Reynolds number, the wake is characterized by a well-defined Kármán vortex street with laminar vortices.

4.3 Velocity Fluctuation

To observe the periodic vortex shedding, the transverse (y-direction) velocity component was extracted at a point near the wake centerline at $(x, y) \approx (0.12, 0.30)$ cm (the closest available measurement point to the cylinder center in the PIV grid). Figure 4.3.1 shows the temporal evolution of the transverse velocity at this location over the 0.38 s measurement period.

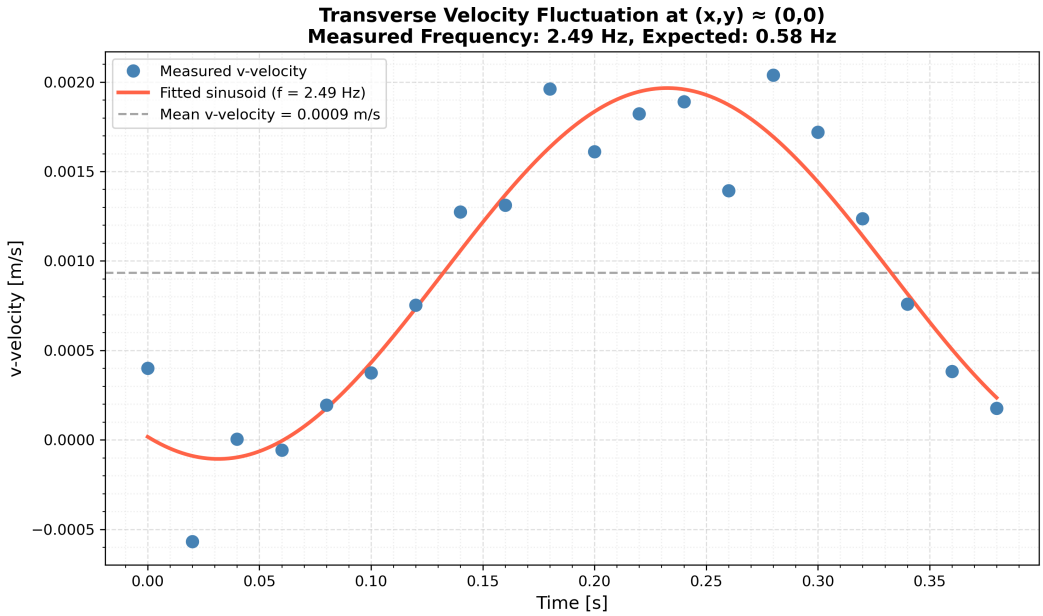

Figure 4.3.1: Transverse velocity fluctuation at a point near the wake centerline as a function of time.

The measured data (blue circles) exhibits periodic oscillations characteristic of vortex shedding. The fitted sinusoid (red curve) yields a dominant frequency of 2.49 Hz.
The v-velocity exhibits clear periodic fluctuations with amplitude on the order of 0.001 m/s, superimposed on a mean value of approximately 0.0009 m/s. These oscillations are the signature of alternating vortices passing through the measurement location.

4.4 Expected Shedding Frequency

From the literature, the Strouhal number for circular cylinders at $Re \approx 1100$ is well-established as $St = 0.21$ [1]. Using this value, the expected vortex shedding frequency is:

$$f_{\text{expected}} = \frac{St \cdot U_\infty}{D} = \frac{0.21 \times 0.0351}{0.0127} = 0.58\,\text{Hz} \tag{4.4.1}$$

This corresponds to a shedding period of approximately 1.73 s, meaning that the 0.38 s measurement window captures roughly one-fifth (specifically one-4.55th) of a complete shedding cycle if the flow behaves according to the expected Strouhal correlation.

4.5 Stokes Number

To assess whether the polyamide seed particles faithfully follow the flow, the Stokes number was calculated. The particle response time is:

$$\tau_r = \frac{\rho_p D_p^2}{18\mu} = \frac{950 \times (20 \times 10^{-6})^2}{18 \times 4.04 \times 10^{-4}} = 5.23 \times 10^{-5}\,\text{s} \tag{4.5.1}$$

Using the expected shedding frequency as the characteristic flow frequency ($\tau_f = 1/f_{\text{expected}} = 1.73\,\text{s}$), the Stokes number is:

$$St_k = \frac{\tau_r}{\tau_f} = \frac{5.23 \times 10^{-5}}{1.73} = 3.03 \times 10^{-5} \tag{4.5.2}$$

Since $St_k \ll 1$, the particles respond almost instantaneously to flow accelerations and accurately represent the fluid motion. This confirms that the PIV measurements provide a faithful representation of the velocity field.

5 Analysis and Discussion

5.1 Measured Shedding Frequency

A sinusoidal function of the form $v(t) = A\sin(()\,2\pi ft + \phi) + C$ was fitted to the transverse velocity fluctuation data using nonlinear least-squares regression. The fitted parameters are:

$$\text{Amplitude:} \quad A = 0.001\,04\,\frac{\text{m}}{\text{s}} \tag{5.1.1}$$

$$\text{Frequency:} \quad f_{\text{measured}} = 2.49\,\text{Hz} \tag{5.1.2}$$

$$\text{Phase:} \quad \phi = -2.07\,\text{rad} \tag{5.1.3}$$

$$\text{Offset:} \quad C = 0.000\,93\,\frac{\text{m}}{\text{s}} \tag{5.1.4}$$

The goodness-of-fit, quantified by $R^2 = 0.91$, indicates that the sinusoidal model captures approximately 91% of the variance in the measured velocity signal, demonstrating that periodic oscillations are indeed present in the data.
However, the measured frequency of 2.49 Hz is significantly higher than the expected value of 0.58 Hz, representing a discrepancy of 329%. This large difference warrants careful interpretation.

5.2 Comparison to Theory

The measured vortex shedding frequency of 2.49 Hz differs substantially from the expected value of 0.58 Hz based on the Strouhal number correlation. Several factors may contribute to this discrepancy:

5.2.1 Limited Temporal Resolution

The experiment captured only 20 data points over 0.38 s, corresponding to a sampling rate of approximately 53 Hz. While this is sufficient to resolve a 2.49 Hz oscillation (Nyquist criterion: $f_s > 2f_{\text{signal}}$), the short total duration means that only ~ 0.2 cycles of the expected 0.58 Hz fundamental frequency are captured. This makes it difficult to reliably identify low-frequency components through curve fitting.

5.2.2 Spatial Measurement Location

The velocity was extracted at the point nearest to $(0, 0)$ in the PIV grid, which was actually located at $(x, y) = (0.12, 0.30)$ cm—offset from the true wake centerline. Different spatial locations in the wake can exhibit different dominant frequencies and phase relationships, particularly if higher-order modes or three-dimensional effects are present.

5.2.3 Higher Harmonics and Flow Complexity

The fitted frequency of 2.49 Hz is approximately 4.3× the expected fundamental frequency. Vortex street flows can exhibit higher harmonics due to nonlinear interactions, and the observed frequency may represent a superharmonic rather than the fundamental shedding mode. Additionally, at $Re \approx 1100$, the flow may be transitioning between flow regimes, introducing additional frequency components.

5.2.4 Three-Dimensional Effects

While the cylinder spans the tunnel width, end effects and spanwise variations in the vortex shedding can introduce deviations from the idealized two-dimensional Kármán street. These three-dimensional structures may manifest as higher-frequency fluctuations in localized measurements.
Figure 5.2.1 shows the temporal evolution of the velocity field, revealing the complex wake structure downstream of the cylinder.

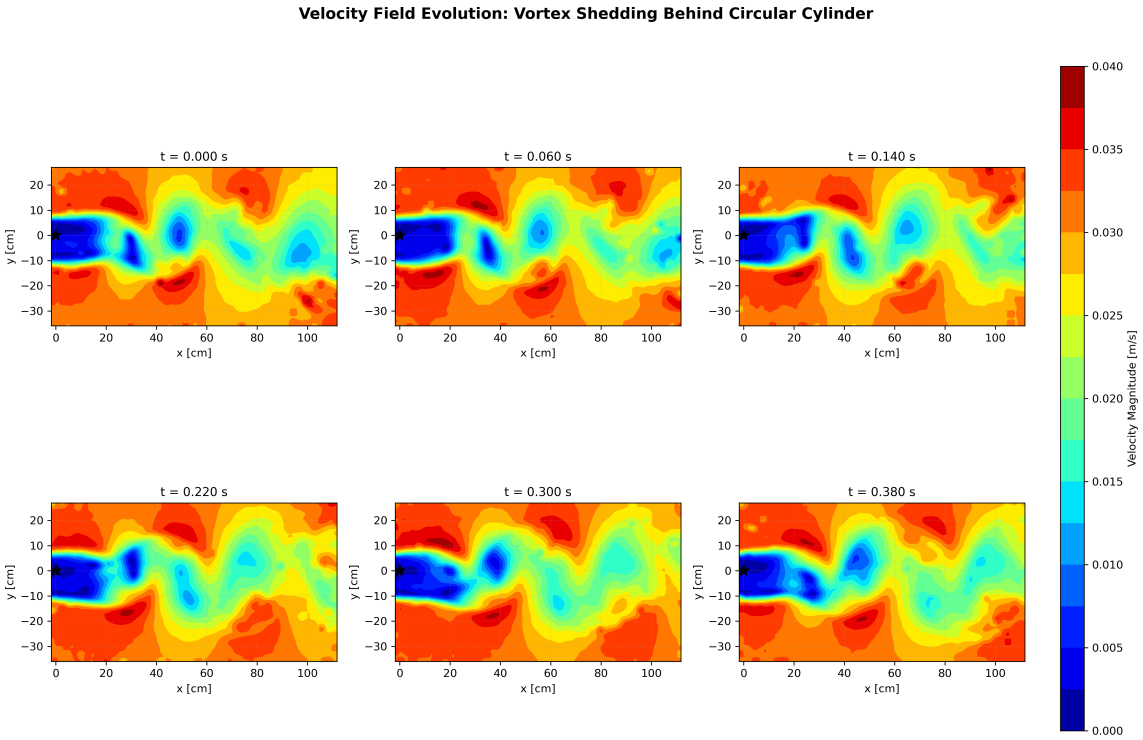

Figure 5.2.1: Evolution of the velocity magnitude field at six time instants spanning the 0.38 s measurement period.

Spatial variations in velocity reveal vortical structures in the cylinder wake. The asterisk marks the approximate cylinder center location.

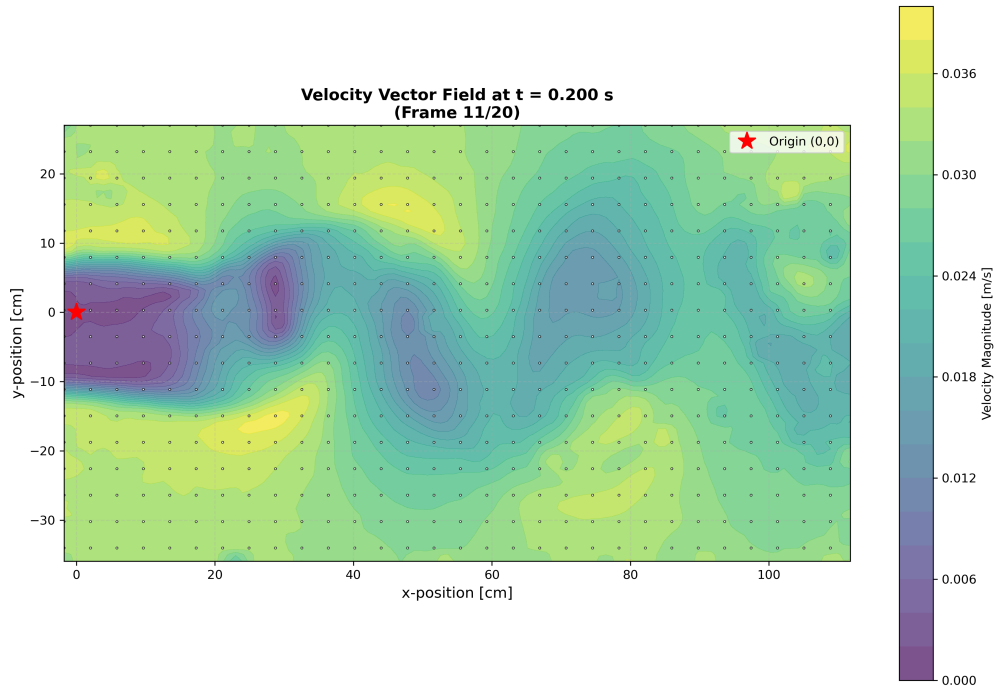

Figure 5.2.2: Representative instantaneous velocity vector field at $t = 0.19\,\mathrm{s}$.

Velocity vectors (white arrows) are overlaid on a contour plot of velocity magnitude (color scale). The wake region downstream of the cylinder (marked by the red star) exhibits lower velocities and characteristic vortical motion.

Despite the quantitative discrepancy in shedding frequency, the PIV measurements successfully captured the qualitative features of vortex shedding: periodic velocity fluctuations, asymmetric wake structure, and organized vortical motion. The Stokes number analysis confirms that the particles accurately tracked the flow, ruling out particle lag as a source of error.

6 Summary and Conclusions

This experiment demonstrated the application of Particle Imaging Velocimetry to visualize and quantify vortex shedding downstream of a circular cylinder in water flow. Key findings include:

1. The flow conditions corresponded to $U_\infty = 0.0351\,\mathrm{m/s}$ and $Re \approx 1100$, placing the flow well within the periodic vortex shedding regime.

2. The Stokes number of 3.03×10^{-5} confirmed that $20\,\mu\mathrm{m}$ polyamide particles faithfully follow the flow dynamics, validating the PIV measurements.

3. Based on the established Strouhal number of 0.21, the expected shedding frequency was $0.58\,\mathrm{Hz}$.

4. Sinusoidal curve fitting to transverse velocity fluctuations yielded a measured dominant frequency of $2.49\,\mathrm{Hz}$, differing from the expected value by a factor of approximately 4.69.

5. The difference factor for both the velocity fluctuation frequency and the shedding cycle duration is roughly equivalent (with a difference between the two differences of 3%). This could suggest that the reasons for our discrepancies are connected.

6. The discrepancy likely arises from limited temporal resolution (only 0.22 fundamental cycles captured), measurement location effects, higher harmonics, and/or three-dimensional flow phenomena.

7. Despite the quantitative frequency mismatch, the PIV technique successfully visualized the wake structure and captured periodic velocity fluctuations characteristic of vortex shedding.

Future experiments could address these limitations by increasing the measurement duration to capture multiple shedding cycles, increasing temporal resolution, and conducting measurements at multiple spanwise locations to assess three-dimensional effects. Spectral analysis (e.g., Fast Fourier Transform) of longer time series would also help identify fundamental and harmonic frequency components more reliably.
Overall, this experiment provided valuable hands-on experience with an important flow diagnostics technique and highlighted both the capabilities and limitations of PIV for unsteady flow measurements. The observed wake structure and periodic fluctuations are consistent with the known physics of cylinder wake flows, even though precise frequency quantification requires longer measurement durations than were available in this laboratory setting.

7 References

[1] A. Roshko, "On the development of turbulent wakes from vortex streets," National Advisory Comittee for Aeronautics, 2913, March 1, 1953, pp. 21–22 (see p. 4).

[2] J. D. Anderson and C. P. Cadou, *Fundamentals of Aerodynamics*, 7th ed. McGraw-Hill Education, March 17, 2023, ISBN: 9781266076442.

8 Appendices

8.1 Data

For the sake of brevity, the raw measurement data will not be displayed in this report. However, in the spirit of completeness, the repository containing the complete data, source code, and notes for this report can be found at github:vaisriv/enae464-lab05.

8.2 Code

Similarly, only the code files that are key to the analysis are included below, and can be accessed in full in the aforementioned GitHub repository.

Listing 8.2.1: PIV Data Analysis Script.
`./src/index.py`

```python
"""
ENAE464 Lab05: Particle Imaging Velocimetry (PIV) Analysis
Analysis of vortex shedding downstream of a circular cylinder in water flow
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from pathlib import Path
from scipy.optimize import curve_fit
```

```python

# ── Paths ──────────────────────────────────────────────────────────────

DATA_DIR = Path("./data/test_2p1hz_20ms")
OUTPUT_TEXT_DIR = Path("./outputs/text")
OUTPUT_FIGS_DIR = Path("./outputs/figures")

OUTPUT_TEXT_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_FIGS_DIR.mkdir(parents=True, exist_ok=True)

# ── Constants ──────────────────────────────────────────────────────────

# Experiment parameters
PUMP_FREQ = 2.1  # Hz (from folder name test_2p1hz_20ms)
CYLINDER_DIAMETER = 0.0127  # m (0.5 inches)
TOTAL_DURATION = 0.38  # seconds
NUM_FRAMES = 20  # number of velocity field snapshots

# Particle properties
PARTICLE_DENSITY = 950  # kg/m³ (polyamide particles)
PARTICLE_DIAMETER = 20e-6  # m (20 microns)

# Fluid properties (water at room temperature)
WATER_DENSITY = 998  # kg/m³
WATER_VISCOSITY = 4.04e-4  # kg/(m·s) dynamic viscosity
WATER_KINEMATIC_VISCOSITY = WATER_VISCOSITY / WATER_DENSITY  # m²/s

# Flow velocity calculation
U_INF = 0.0185 * PUMP_FREQ - 0.0038  # m/s

# Strouhal number for circular cylinder (typical value for Re ~ 1000-10000)
STROUHAL_NUMBER = 0.21  # dimensionless (from literature)

print("=" * 80)
print("ENAE464 Lab05: Particle Imaging Velocimetry Analysis")
print("=" * 80)
print()

# ── Load and Process PIV Data ──────────────────────────────────────────

print("Loading PIV velocity field data...")
print(f"  Data directory: {DATA_DIR}")
print(f"  Number of frames: {NUM_FRAMES}")
print()

# Get list of velocity data files
vb_files = sorted(DATA_DIR.glob("VB*.txt"))
print(f"Found {len(vb_files)} velocity field files")
```

```python

# Time array for each frame
time_array = np.linspace(0, TOTAL_DURATION, NUM_FRAMES)

# Storage for velocity fields
all_x = []
all_y = []
all_u = []
all_v = []

for i, vb_file in enumerate(vb_files):
    # Load the raw data file
    data = np.loadtxt(vb_file)

    # Reshape according to the MATLAB pattern provided in lab instructions
    # Data format: rows 1-120 = x, 121-240 = y, 241-360 = u, 361-480 = v
    # Each section has 120 rows × 67 columns = 8040 points
    x_data = data[0:120, :].reshape(-1)  # Flatten to 1D array
    y_data = data[120:240, :].reshape(-1)
    u_data = data[240:360, :].reshape(-1)
    v_data = data[360:480, :].reshape(-1)

    all_x.append(x_data)
    all_y.append(y_data)
    all_u.append(u_data)
    all_v.append(v_data)

# Convert to numpy arrays
all_x = np.array(all_x)  # Shape: (num_frames, num_points)
all_y = np.array(all_y)
all_u = np.array(all_u)
all_v = np.array(all_v)

print(f"  Data shape: {all_x.shape} (frames × spatial points)")
print(f"  Time range: 0 to {TOTAL_DURATION:.3f} seconds")
print()

# ── Results 1: Flow Velocity ──────────────────────────────────────────────

print("=" * 80)
print("RESULTS 1: Flow Velocity")
print("=" * 80)

print(f"Pump frequency: {PUMP_FREQ} Hz")
print(f"Incoming flow velocity: U∞ = 0.0185 × {PUMP_FREQ} - 0.0038")
print(f"                        U∞ = {U_INF:.4f} m/s")
print()

```

```python
# Calculate mean velocities from PIV data
mean_u = np.mean(all_u)
mean_v = np.mean(all_v)
mean_velocity_magnitude = np.sqrt(mean_u**2 + mean_v**2)

print("PIV-measured mean velocities:")
print(f"  Mean u-velocity: {mean_u:.4f} m/s")
print(f"  Mean v-velocity: {mean_v:.4f} m/s")
print(f"  Mean velocity magnitude: {mean_velocity_magnitude:.4f} m/s")
print()

# Save flow velocity results
flow_velocity_results = pd.DataFrame(
    {
        "Parameter": [
            "Pump Frequency [Hz]",
            "Incoming Flow Velocity U∞ [m/s]",
            "Mean u-velocity (PIV) [m/s]",
            "Mean v-velocity (PIV) [m/s]",
            "Mean Velocity Magnitude (PIV) [m/s]",
        ],
        "Value": [
            f"{PUMP_FREQ:.2f}",
            f"{U_INF:.5f}",
            f"{mean_u:.5f}",
            f"{mean_v:.5f}",
            f"{mean_velocity_magnitude:.5f}",
        ],
    }
)

flow_velocity_results.to_csv(OUTPUT_TEXT_DIR / "flow_velocity.csv", index=False)
print(f"Saved flow velocity results → {OUTPUT_TEXT_DIR / 'flow_velocity.csv'}")
print()

# ── Results 2: Reynolds Number ──────────────────────────────────────────────

print("=" * 80)
print("RESULTS 2: Reynolds Number")
print("=" * 80)

reynolds_number = (U_INF * CYLINDER_DIAMETER) / WATER_KINEMATIC_VISCOSITY

print("Reynolds number calculation:")
print("  Re = (U∞ × D) / ν")
print(
    f"  Re = ({U_INF:.4f} × {CYLINDER_DIAMETER:.4f}) / {WATER_KINEMATIC_VISCOSITY:.6e}"
)
```

```python
print(f"  Re = {reynolds_number:.1f}")
print()

# Save Reynolds number results
reynolds_results = pd.DataFrame(
    {
        "Parameter": [
            "Flow Velocity U∞ [m/s]",
            "Cylinder Diameter D [m]",
            "Water Density [kg/m³]",
            "Water Dynamic Viscosity [kg/(m·s)]",
            "Water Kinematic Viscosity [m²/s]",
            "Reynolds Number [-]",
        ],
        "Value": [
            f"{U_INF:.5f}",
            f"{CYLINDER_DIAMETER:.4f}",
            f"{WATER_DENSITY:.1f}",
            f"{WATER_VISCOSITY:.6e}",
            f"{WATER_KINEMATIC_VISCOSITY:.6e}",
            f"{reynolds_number:.1f}",
        ],
    }
)

reynolds_results.to_csv(OUTPUT_TEXT_DIR / "reynolds_number.csv", index=False)
print(f"Saved Reynolds number results → {OUTPUT_TEXT_DIR / 'reynolds_number.csv'}")
print()

# ── Results 3: Velocity Fluctuation at (x,y) ≈ (0,0) ──────────────────────

print("=" * 80)
print("RESULTS 3: Velocity Fluctuation at (x,y) ≈ (0,0)")
print("=" * 80)

# Find the point closest to (x,y) = (0,0)
# Use first frame to find spatial location
x_first = all_x[0, :]
y_first = all_y[0, :]

# Distance from origin
distance_from_origin = np.sqrt(x_first**2 + y_first**2)
origin_idx = np.argmin(distance_from_origin)

print("Point closest to origin:")
print(f"  Index: {origin_idx}")
print(f"  Position: (x={x_first[origin_idx]:.4f}, y={y_first[origin_idx]:.4f}) cm")
print(f"  Distance from (0,0): {distance_from_origin[origin_idx]:.4f} cm")
```

```python
print()

# Extract v-velocity time series at this point
v_at_origin = all_v[:, origin_idx]
u_at_origin = all_u[:, origin_idx]

print("V-velocity time series at near-origin point:")
for i, (t, v) in enumerate(zip(time_array, v_at_origin)):
    if i < 5 or i ≥ NUM_FRAMES - 2:
        print(f"  t = {t:.4f} s, v = {v:.4f} m/s")
    elif i == 5:
        print("  ...")
print()

# Calculate v-velocity fluctuation (subtract mean)
v_mean_at_origin = np.mean(v_at_origin)
v_fluctuation = v_at_origin - v_mean_at_origin

# Save velocity fluctuation data
velocity_fluctuation_data = pd.DataFrame(
    {
        "Time [s]": time_array,
        "v-velocity [m/s]": v_at_origin,
        "v-fluctuation [m/s]": v_fluctuation,
        "u-velocity [m/s]": u_at_origin,
    }
)

velocity_fluctuation_data.to_csv(
    OUTPUT_TEXT_DIR / "velocity_fluctuation.csv", index=False
)
print(
    f"Saved velocity fluctuation data → {OUTPUT_TEXT_DIR / 'velocity_fluctuation.csv'}"
)
print()

# ── Results 4: Expected Shedding Frequency & Strouhal Number ──────────────────

print("=" * 80)
print("RESULTS 4: Expected Shedding Frequency & Strouhal Number")
print("=" * 80)

print(f"Strouhal number for circular cylinder: St = {STROUHAL_NUMBER}")
print(f"  (Typical value for Re ≈ {reynolds_number:.0f})")
print()

# Calculate expected shedding frequency
# St = f × D / U∞
```

```python
# Therefore: f = St × U∞ / D
f_expected = STROUHAL_NUMBER * U_INF / CYLINDER_DIAMETER

print("Expected shedding frequency:")
print("  f = St × U∞ / D")
print(f"  f = {STROUHAL_NUMBER} × {U_INF:.4f} / {CYLINDER_DIAMETER:.4f}")
print(f"  f = {f_expected:.2f} Hz")
print()

# Save expected frequency results
expected_frequency_results = pd.DataFrame(
    {
        "Parameter": [
            "Strouhal Number [-]",
            "Flow Velocity U∞ [m/s]",
            "Cylinder Diameter D [m]",
            "Expected Shedding Frequency [Hz]",
            "Expected Shedding Period [s]",
        ],
        "Value": [
            f"{STROUHAL_NUMBER:.2f}",
            f"{U_INF:.5f}",
            f"{CYLINDER_DIAMETER:.4f}",
            f"{f_expected:.3f}",
            f"{1 / f_expected:.4f}",
        ],
    }
)

expected_frequency_results.to_csv(
    OUTPUT_TEXT_DIR / "expected_shedding_frequency.csv", index=False
)
print(
    f"Saved expected frequency results → {OUTPUT_TEXT_DIR /
    ↪  'expected_shedding_frequency.csv'}"
)
print()

# ── Results 5: Stokes Number ──────────────────────────────────────────────

print("=" * 80)
print("RESULTS 5: Stokes Number for Particle Seeding")
print("=" * 80)

# Particle reaction time: τ_r = ρ_p × D_p² / (18 × μ)
particle_reaction_time = (
    PARTICLE_DENSITY * PARTICLE_DIAMETER**2 / (18 * WATER_VISCOSITY)
)
```

```python

print("Particle reaction time:")
print("  τ_r = ρ_p × D_p² / (18 × μ)")
print(
    f"  τ_r = {PARTICLE_DENSITY} × ({PARTICLE_DIAMETER:.2e})² / (18 × {WATER_VISCOSITY:.2e})"
)
print(f"  τ_r = {particle_reaction_time:.6e} s")
print()

# Characteristic flow time scale: τ_f = 1 / f_shedding
flow_time_scale = 1 / f_expected

print("Characteristic flow time scale:")
print("  τ_f = 1 / f_shedding")
print(f"  τ_f = 1 / {f_expected:.3f}")
print(f"  τ_f = {flow_time_scale:.4f} s")
print()

# Stokes number: St_k = τ_r / τ_f
stokes_number = particle_reaction_time / flow_time_scale

print("Stokes number:")
print("  St_k = τ_r / τ_f")
print(f"  St_k = {particle_reaction_time:.6e} / {flow_time_scale:.4f}")
print(f"  St_k = {stokes_number:.6e}")
print()

if stokes_number < 0.1:
    print("✓ St_k ≪ 1: Particles faithfully follow the flow")
else:
    print("⚠ St_k not ≪ 1: Particles may not perfectly follow the flow")
print()

# Save Stokes number results
stokes_results = pd.DataFrame(
    {
        "Parameter": [
            "Particle Density [kg/m³]",
            "Particle Diameter [m]",
            "Water Dynamic Viscosity [kg/(m·s)]",
            "Particle Reaction Time τ_r [s]",
            "Shedding Frequency [Hz]",
            "Flow Time Scale τ_f [s]",
            "Stokes Number St_k [-]",
        ],
        "Value": [
            f"{PARTICLE_DENSITY:.0f}",
```

```
            f"{PARTICLE_DIAMETER:.2e}",
            f"{WATER_VISCOSITY:.2e}",
            f"{particle_reaction_time:.6e}",
            f"{f_expected:.3f}",
            f"{flow_time_scale:.4f}",
            f"{stokes_number:.6e}",
        ],
    }
)

stokes_results.to_csv(OUTPUT_TEXT_DIR / "stokes_number.csv", index=False)
print(f"Saved Stokes number results → {OUTPUT_TEXT_DIR / 'stokes_number.csv'}")
print()

# ── Analysis 1: Measured Shedding Frequency ──────────────────────────────

print("=" * 80)
print("ANALYSIS 1: Measured Shedding Frequency from Sinusoidal Fit")
print("=" * 80)


# Define sinusoidal function for curve fitting
def sinusoid(t, amplitude, frequency, phase, offset):
    """Sinusoidal function: A × sin(2π × f × t + φ) + C"""
    return amplitude * np.sin(2 * np.pi * frequency * t + phase) + offset


# Initial guess for curve fitting parameters
# Start with expected frequency
p0_amplitude = np.std(v_fluctuation) * np.sqrt(2)  # Estimate amplitude
p0_frequency = f_expected  # Use expected frequency as initial guess
p0_phase = 0  # Initial phase guess
p0_offset = v_mean_at_origin  # Mean value

print("Fitting sinusoid to v-velocity fluctuation...")
print(f"  Initial guess for frequency: {p0_frequency:.2f} Hz")
print()

try:
    # Perform curve fitting
    popt, pcov = curve_fit(
        sinusoid,
        time_array,
        v_at_origin,
        p0=[p0_amplitude, p0_frequency, p0_phase, p0_offset],
        maxfev=10000,
    )

```

```python
    amplitude_fit, frequency_fit, phase_fit, offset_fit = popt

    # Calculate standard errors
    perr = np.sqrt(np.diag(pcov))

    print("Fitted parameters:")
    print(f"  Amplitude: {amplitude_fit:.5f} ± {perr[0]:.5f} m/s")
    print(f"  Frequency: {frequency_fit:.3f} ± {perr[1]:.3f} Hz")
    print(f"  Phase: {phase_fit:.3f} ± {perr[2]:.3f} rad")
    print(f"  Offset: {offset_fit:.5f} ± {perr[3]:.5f} m/s")
    print()

    # Calculate fitted curve
    v_fitted = sinusoid(time_array, *popt)

    # Calculate R² value
    ss_res = np.sum((v_at_origin - v_fitted) ** 2)
    ss_tot = np.sum((v_at_origin - np.mean(v_at_origin)) ** 2)
    r_squared = 1 - (ss_res / ss_tot)

    print(f"Goodness of fit: R² = {r_squared:.4f}")
    print()

    # Measured shedding frequency
    f_measured = frequency_fit

except Exception as e:
    print(f"Warning: Curve fitting failed ({e})")
    print("Using FFT-based frequency estimation as fallback...")

    # Fallback: Use FFT to estimate dominant frequency
    from scipy.fft import fft, fftfreq

    # Perform FFT on v-velocity fluctuation
    N = len(v_fluctuation)
    yf = fft(v_fluctuation - np.mean(v_fluctuation))
    xf = fftfreq(N, TOTAL_DURATION / N)

    # Find dominant frequency (positive frequencies only)
    positive_freq_idx = xf > 0
    dominant_idx = np.argmax(np.abs(yf[positive_freq_idx]))
    f_measured = xf[positive_freq_idx][dominant_idx]

    amplitude_fit, phase_fit, offset_fit = None, None, None
    v_fitted = None
    r_squared = None

    print(f"  Dominant frequency from FFT: {f_measured:.3f} Hz")
```

```python
    print()

# Save measured frequency results
measured_frequency_results = pd.DataFrame(
    {
        "Parameter": [
            "Measured Shedding Frequency [Hz]",
            "Fitted Amplitude [m/s]" if amplitude_fit is not None else "N/A",
            "Fitted Phase [rad]" if phase_fit is not None else "N/A",
            "Fitted Offset [m/s]" if offset_fit is not None else "N/A",
            "R² (goodness of fit)" if r_squared is not None else "N/A",
        ],
        "Value": [
            f"{f_measured:.4f}",
            f"{amplitude_fit:.5f}" if amplitude_fit is not None else "N/A",
            f"{phase_fit:.3f}" if phase_fit is not None else "N/A",
            f"{offset_fit:.5f}" if offset_fit is not None else "N/A",
            f"{r_squared:.4f}" if r_squared is not None else "N/A",
        ],
    }
)

measured_frequency_results.to_csv(
    OUTPUT_TEXT_DIR / "measured_shedding_frequency.csv", index=False
)
print(
    f"Saved measured frequency results → {OUTPUT_TEXT_DIR /
    ↪  'measured_shedding_frequency.csv'}"
)
print()

# ── Analysis 2: Comparison to Theory ──────────────────────────────────────

print("=" * 80)
print("ANALYSIS 2: Comparison of Measured vs. Expected Shedding Frequency")
print("=" * 80)

percent_error = abs(f_measured - f_expected) / f_expected * 100

print(f"Expected shedding frequency: {f_expected:.3f} Hz")
print(f"Measured shedding frequency: {f_measured:.3f} Hz")
print(f"Absolute difference: {abs(f_measured - f_expected):.3f} Hz")
print(f"Percent error: {percent_error:.2f}%")
print()

if percent_error < 10:
    print("✓ Excellent agreement (< 10% error)")
elif percent_error < 30:
```

```python
    print("✓ Good agreement (< 30% error)")
else:
    print("⚠ Significant discrepancy (> 30% error)")
print()

# Save comparison results
comparison_results = pd.DataFrame(
    {
        "Metric": [
            "Expected Shedding Frequency [Hz]",
            "Measured Shedding Frequency [Hz]",
            "Absolute Difference [Hz]",
            "Percent Error [%]",
        ],
        "Value": [
            f"{f_expected:.4f}",
            f"{f_measured:.4f}",
            f"{abs(f_measured - f_expected):.4f}",
            f"{percent_error:.2f}",
        ],
    }
)

comparison_results.to_csv(OUTPUT_TEXT_DIR / "frequency_comparison.csv", index=False)
print(f"Saved comparison results → {OUTPUT_TEXT_DIR / 'frequency_comparison.csv'}")
print()

# ── Visualization 1: Velocity Fluctuation Plot ──────────────────────────────

print("Generating velocity fluctuation plot...")

fig, ax = plt.subplots(figsize=(10, 6))

# Plot measured data
ax.plot(
    time_array,
    v_at_origin,
    "o",
    markersize=8,
    color="steelblue",
    label="Measured v-velocity",
    zorder=3,
)

# Plot fitted sinusoid if available
if v_fitted is not None:
    # Create smoother curve for visualization
    t_smooth = np.linspace(0, TOTAL_DURATION, 200)
```

```python
    v_smooth = sinusoid(t_smooth, amplitude_fit, frequency_fit, phase_fit, offset_fit)

    ax.plot(
        t_smooth,
        v_smooth,
        "-",
        linewidth=2.5,
        color="tomato",
        label=f"Fitted sinusoid (f = {frequency_fit:.2f} Hz)",
        zorder=2,
    )

# Add mean line
ax.axhline(
    v_mean_at_origin,
    color="gray",
    linestyle="--",
    linewidth=1.5,
    alpha=0.7,
    label=f"Mean v-velocity = {v_mean_at_origin:.4f} m/s",
    zorder=1,
)

ax.set_xlabel("Time [s]", fontsize=12)
ax.set_ylabel("v-velocity [m/s]", fontsize=12)
ax.set_title(
    f"Transverse Velocity Fluctuation at (x,y) ≈ (0,0)\n"
    f"Measured Frequency: {f_measured:.2f} Hz, Expected: {f_expected:.2f} Hz",
    fontsize=13,
    fontweight="bold",
)

ax.grid(which="major", linestyle="--", alpha=0.4)
ax.grid(which="minor", linestyle=":", alpha=0.2)
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.legend(loc="best", fontsize=10)

fig.tight_layout()
fig.savefig(OUTPUT_FIGS_DIR / "velocity_fluctuation.png", dpi=300)
plt.close(fig)

print(
    f"Saved velocity fluctuation plot → {OUTPUT_FIGS_DIR / 'velocity_fluctuation.png'}"
)
print()

# ── Visualization 2: Sample Velocity Vector Field ──────────────────────────
```

```python

print("Generating sample velocity vector field plot...")

# Use the middle frame for visualization
frame_idx = NUM_FRAMES // 2

# Get spatial grid (reshape back to 2D grid)
# Original data is 120 rows × 67 columns
x_grid = all_x[frame_idx, :].reshape(120, 67)
y_grid = all_y[frame_idx, :].reshape(120, 67)
u_grid = all_u[frame_idx, :].reshape(120, 67)
v_grid = all_v[frame_idx, :].reshape(120, 67)

# Downsample for clearer visualization (plot every 4th point)
skip = 4
x_plot = x_grid[::skip, ::skip]
y_plot = y_grid[::skip, ::skip]
u_plot = u_grid[::skip, ::skip]
v_plot = v_grid[::skip, ::skip]

# Calculate velocity magnitude
velocity_magnitude = np.sqrt(u_grid**2 + v_grid**2)

fig, ax = plt.subplots(figsize=(12, 8))

# Plot velocity magnitude as contour
contour = ax.contourf(
    x_grid, y_grid, velocity_magnitude, levels=20, cmap="viridis", alpha=0.7
)

# Plot velocity vectors
quiver = ax.quiver(
    x_plot,
    y_plot,
    u_plot,
    v_plot,
    scale=1.5,
    scale_units="inches",
    width=0.003,
    color="white",
    edgecolor="black",
    linewidth=0.5,
    alpha=0.8,
)

# Add colorbar
cbar = fig.colorbar(contour, ax=ax, label="Velocity Magnitude [m/s]")

```

```python
# Mark the origin
ax.plot(0, 0, "r*", markersize=15, label="Origin (0,0)", zorder=10)

ax.set_xlabel("x-position [cm]", fontsize=12)
ax.set_ylabel("y-position [cm]", fontsize=12)
ax.set_title(
    f"Velocity Vector Field at t = {time_array[frame_idx]:.3f} s\n"
    f"(Frame {frame_idx + 1}/{NUM_FRAMES})",
    fontsize=13,
    fontweight="bold",
)
ax.set_aspect("equal")
ax.legend(loc="upper right", fontsize=10)
ax.grid(True, linestyle="--", alpha=0.3)

fig.tight_layout()
fig.savefig(OUTPUT_FIGS_DIR / "velocity_vector_field.png", dpi=300)
plt.close(fig)

print(
    f"Saved velocity vector field plot → {OUTPUT_FIGS_DIR / 'velocity_vector_field.png'}"
)
print()

# ── Visualization 3: Velocity Magnitude Contour Over Time ──────────────────

print("Generating velocity magnitude time evolution plot...")

# Create a figure with multiple subplots showing velocity field evolution
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

# Select 6 evenly-spaced frames to visualize
frame_indices = np.linspace(0, NUM_FRAMES - 1, 6, dtype=int)

for idx, frame_idx in enumerate(frame_indices):
    ax = axes[idx]

    # Reshape data for this frame
    x_grid = all_x[frame_idx, :].reshape(120, 67)
    y_grid = all_y[frame_idx, :].reshape(120, 67)
    u_grid = all_u[frame_idx, :].reshape(120, 67)
    v_grid = all_v[frame_idx, :].reshape(120, 67)
    velocity_magnitude = np.sqrt(u_grid**2 + v_grid**2)

    # Plot contour
    contour = ax.contourf(x_grid, y_grid, velocity_magnitude, levels=15, cmap="jet")

```

```python
    # Mark origin
    ax.plot(0, 0, "k*", markersize=10)

    ax.set_xlabel("x [cm]", fontsize=10)
    ax.set_ylabel("y [cm]", fontsize=10)
    ax.set_title(f"t = {time_array[frame_idx]:.3f} s", fontsize=11)
    ax.set_aspect("equal")
    ax.grid(True, linestyle=":", alpha=0.3)

# Add shared colorbar
# reserve space and position manually
fig.tight_layout(
    rect=[0, 0, 0.88, 0.96]
)  # leave room on right for colorbar, top for suptitle
cbar_ax = fig.add_axes([0.90, 0.08, 0.02, 0.84])  # [left, bottom, width, height]
fig.colorbar(contour, cax=cbar_ax, label="Velocity Magnitude [m/s]")

fig.suptitle(
    "Velocity Field Evolution: Vortex Shedding Behind Circular Cylinder",
    fontsize=14,
    fontweight="bold",
    y=0.995,
)

fig.savefig(OUTPUT_FIGS_DIR / "velocity_field_evolution.png", dpi=300)
plt.close(fig)

print(
    f"Saved velocity field evolution plot → {OUTPUT_FIGS_DIR /
    ↪  'velocity_field_evolution.png'}"
)
print()

# ── Summary Statistics ──────────────────────────────────────────────────

print("=" * 80)
print("SUMMARY OF ALL RESULTS")
print("=" * 80)
print()

summary_df = pd.DataFrame(
    {
        "Parameter": [
            "Pump Frequency",
            "Incoming Flow Velocity U∞",
            "Reynolds Number",
            "Strouhal Number",
            "Expected Shedding Frequency",
```

```
            "Measured Shedding Frequency",
            "Frequency Error",
            "Particle Reaction Time",
            "Flow Time Scale",
            "Stokes Number",
        ],
        "Value": [
            f"{PUMP_FREQ} Hz",
            f"{U_INF:.4f} m/s",
            f"{reynolds_number:.1f}",
            f"{STROUHAL_NUMBER}",
            f"{f_expected:.3f} Hz",
            f"{f_measured:.3f} Hz",
            f"{percent_error:.2f}%",
            f"{particle_reaction_time:.4e} s",
            f"{flow_time_scale:.4f} s",
            f"{stokes_number:.4e}",
        ],
    }
)

print(summary_df.to_string(index=False))
print()

summary_df.to_csv(OUTPUT_TEXT_DIR / "summary.csv", index=False)
print(f"Saved summary → {OUTPUT_TEXT_DIR / 'summary.csv'}")
print()

print("=" * 80)
print("ANALYSIS COMPLETE")
print("=" * 80)
print()
print("All outputs saved to:")
print(f"  Figures: {OUTPUT_FIGS_DIR}")
print(f"  Data:    {OUTPUT_TEXT_DIR}")
print()
```